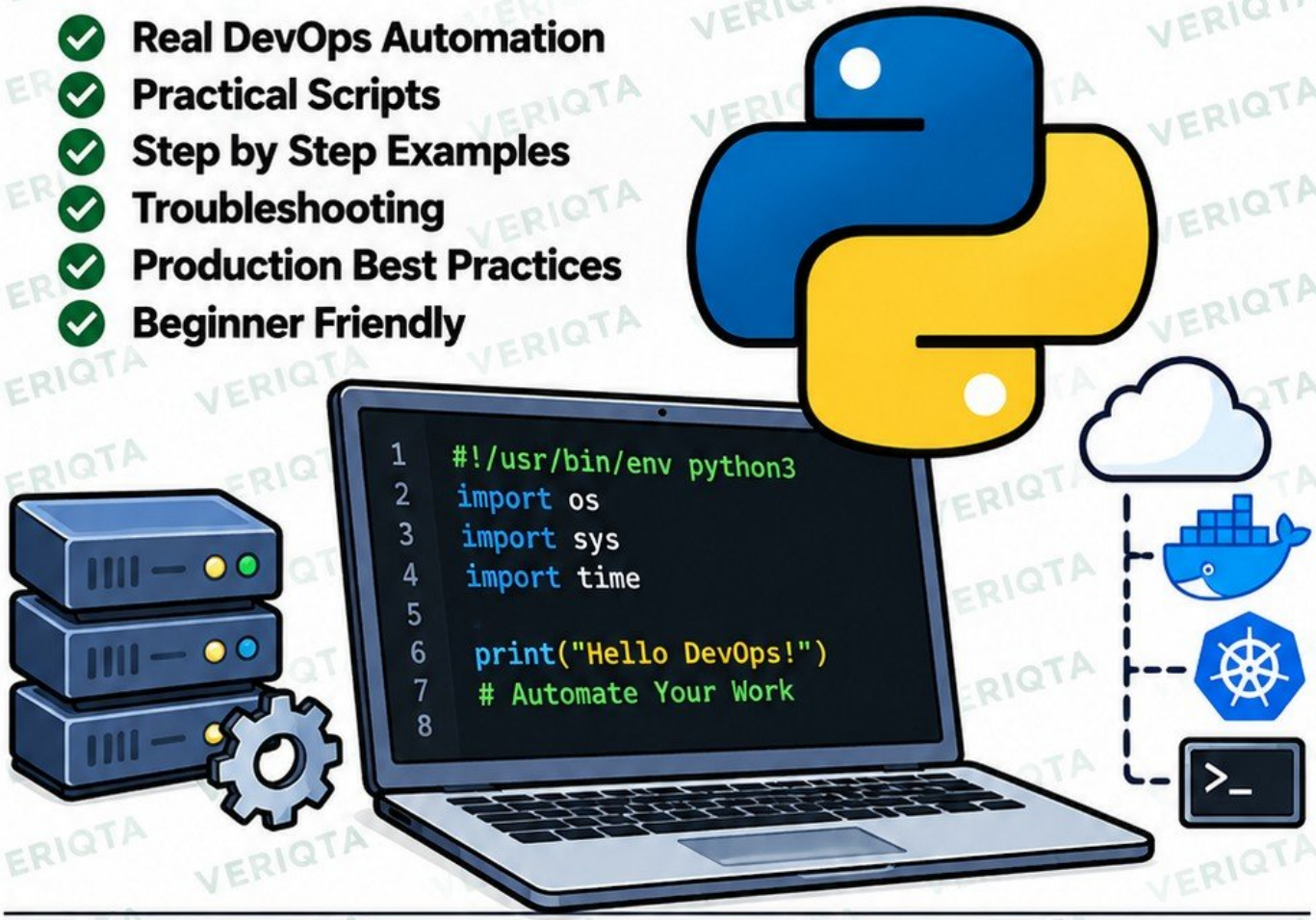
VERIQTA

Python for DevOps: Your First 10 Scripts

- Real DevOps Automation
- Practical Scripts
- Step by Step Examples
- Troubleshooting
- Production Best Practices
- Beginner Friendly



How the 10 Scripts Fit Into Real DevOps Work

Script	What It Does	Engineering Responsibility
01 System Health	Checks disk usage, system load and key components to see if the server is healthy.	Host monitoring
02 Large File Finder	Finds large files that may be filling up disk space.	Storage investigation
03 Log Error Detector	Scans application logs and finds ERROR and CRITICAL messages.	Incident investigation
04 HTTP Health Check	Checks if an application is responding over HTTP and returns the correct status.	Application availability
05 Configuration Backup	Creates timestamped backups of important configuration files.	Change safety and recovery
06 Service Status	Checks the status of Linux services using systemctl.	Linux operations
07 Configuration Validator	Validates JSON configuration files before deployment.	Deployment safety
08 Docker Status	Checks Docker containers and their running states.	Container operations
09 Inventory Report	Reads a CSV file and generates a clean infrastructure inventory report.	Infrastructure management
10 Deployment Verifier	Verifies a deployment by checking application health, version and key dependencies.	Release validation

Why DevOps Engineers Use Python

1. What is a Python Script?

A Python script is a file with Python code (usually ending with `.py`) that tells the computer what to do.

```
# hello.py
print("Hello DevOps!")
```

You run it from the terminal:

```
python3 hello.py
```

2. Automation vs Manual Operations

Manual Operations

- Run commands one by one
- Takes more time
- Higher chance of human error
- Hard to repeat consistently
- Not scalable



You do the same tasks every time.

Automation with Python

- Runs tasks automatically
- Fast and consistent
- Reduces human error
- Easy to repeat and modify
- Can handle many servers
- Works with APIs and tools



One script does the same tasks every time.

3. Where Python Fits in DevOps

Python works alongside many DevOps tools:

Linux
Python runs on Linux and automates system tasks.

Bash
Bash is great for simple tasks. Python is better for complex logic.

Git
Automate repository tasks using Python.

CI/CD
Use Python scripts in pipelines for testing, validation and deployment.

Cloud APIs
Use Python to interact with AWS, Azure or GCP services.

Monitoring
Collect metrics, check health and send alerts using Python.

Python connects all these tools and helps you automate real DevOps work.

Python

4. Real Example: Check Application Health

Manual Way (Slow and Repetitive)

1. SSH to the server
2. Run `curl` to check the website
3. Run `systemctl` to check the service
4. Check the logs manually
5. Repeat this for every server

Automated with Python (Fast and Consistent)

- Checks the website URL
- Checks if the service is running
- Shows a clear status report
- Can check many servers
- Can be scheduled to run automatically

5. Example Output

```
$ python3 health_check.py
Checking application health...
Website:    OK (HTTP 200)
Service:    RUNNING
Disk Usage: 42%
Status:     HEALTHY
```

6. Key Benefits

- Automates repetitive tasks
- Saves time
- Reduces human error
- Works with cloud and DevOps tools
- Helpful for monitoring and reporting
- Easy to customize
- Used in real production environments

7. Junior Engineer Insight



Automate repeatable checks first.

Do not automate a process you cannot explain or verify.

Set Up Your First Python Automation Workspace

1. Install and Verify Python 3

Most Linux and macOS systems already have Python 3. Check the version:

```
$ python3 --version
Python 3.11.4
```

If not installed (Ubuntu / Debian):

```
$ sudo apt update
$ sudo apt install python3 python3-venv
```

If not installed (Amazon Linux / RHEL):

```
$ sudo yum install python3
```

2. python3 vs pip

python3

- Runs Python scripts.
- Starts the Python interpreter.
- Executes .py files.
- Comes with the Python runtime.



```
python3 script.py
```

pip

- Installs Python packages (libraries).
- Does not run your script.
- Used to install tools like requests, boto3, etc.
- Works with Python 3 (usually pip3).



```
pip install requests
```

3. Create a Project Directory

Keep your automation scripts in a dedicated folder.

```
$ mkdir python-devops
$ cd python-devops
$ ls
```



python-devops

4. Understand .py Files

- A .py file is a plain text file that contains Python code.
- You can create and edit it using VS Code, nano, or any text editor.
- The file extension must be .py.
- Give your file a meaningful name.



hello_devops.py

5. Create and Run Your First Script

1. Create the file:

```
$ nano hello_devops.py
```

2. Add this code:

```
print("Hello DevOps!")
print("Python is ready
for automation!")
```

3. Run the script:

```
$ python3 hello_devops.py
```

Expected output:

```
Hello DevOps!
Python is ready
for automation!
```

6. Use a Virtual Environment

A virtual environment keeps your project dependencies separate from the system Python.

```
$ python3 -m venv .venv
```

Activate it:

Linux / macOS	Windows (PowerShell)
$ source .venv/bin/activate	$.\.venv\Scripts\Activate.ps1

After activation, your prompt will usually show (.venv):

```
(.venv) $
```

7. Common Error: python3: command not found

Why it happens:

- Python 3 is not installed.
- The system only has python (not python3).
- Python is installed but not in your PATH.
- You are using an old Linux version.
- You are in a restricted environment.

How to fix:

1. Install Python 3.
2. Use python instead of python3 (if only python is available).
3. Check the path: which python3.
4. Reopen your terminal after installation.

8. Practice

- ✓ Create the file hello_devops.py.
- ✓ Run the script and check the output.
- ✓ Create and activate a virtual environment.
- ✓ Install a package using pip (e.g. requests).
- ✓ Try running the script inside the virtual environment.
- ✓ Identify why python3: command not found may occur on your system.

The Python Essentials Behind Automation

1. Variables (Store Data)

Variables hold information that can be used in your script.

```
server = "web-01"
disk_threshold = 80
service = "nginx"
is_running = True
```

- Use meaningful names.
- You can change the value later.
- Variables make your script flexible.

2. Strings and Numbers

Strings are text. Numbers are used for calculations:

```
app_name = "myapp"
version = "1.0.0"
port = 8080
threshold = 80
message = "App is running"
```

- Strings use quotes.
- Numbers do not use quotes.
- You can combine them:

```
print(app_name + " v" + version)
```

3. Lists (Multiple Items)

Lists store multiple values.

```
servers = ["web-01", "web-02", "web-03"]

for server in servers:
    print("Checking:", server)
```

- Use lists to work with many servers, services or files.
- You can loop through each item using a for loop.

4. Dictionaries (Key and Value)

Dictionaries store related information.

```
server = {
    "name": "web-01",
    "ip": "10.0.0.10",
    "env": "prod",
    "status": "running"
}
```



- Use keys to access values.
- Useful for structured data like server inventory or service status.

5. if Statements (Make Decisions)

Run different code based on a condition.

```
disk_usage = 85
if disk_usage > 80:
    print("High disk usage!")
else:
    print("Disk usage is OK.")
```

- Common in monitoring and alerting.
- Use it to check thresholds, service states, or API responses.

6. Loops (Repeat Tasks)

Use loops to process multiple items.

```
services = ["nginx", "docker", "ssh"]
for service in services:
    print(f"Checking {service}")
```

- Loops help you automate repetitive tasks.
- Useful for checking many servers, files, or services.

7. Functions (Reuse Code)

Functions let you group code and reuse it.

```
def check_disk(usage):
    if usage > 80:
        return "High"
    else:
        return "OK"
```

- Functions make your code clean and reusable.
- You can call the function with different values.

8. Imports (Use Built-in Tools)

Import modules to get extra functionality.

```
import os
import shutil
import time

usage = shutil.disk_usage("/")
print(usage.percent)
```

- Python has many built-in modules.
- Use modules to work with files, system information, APIs, and more.

9. Indentation (Very Important)

Python uses indentation to define blocks of code.

✓
```
if True:
    print("This is correct")
    print("Same block")
```

✗
```
if True:
    print("This will fail")
```

- Use 4 spaces for indentation.
- Incorrect indentation will cause an error.

10. try/except (Handle Errors)

Use try/except to handle predictable failures.

```
try:
    value = int("100")
    print("Value:", value)
except ValueError:
    print("Invalid number!")
```

- Prevents your script from crashing.
- Handle errors such as file not found, API failures, or invalid input.

11. Real Example: Check Disk Usage

This example uses variables, a function, an if statement and a simulated disk usage value.

```
def check_disk(usage):
    if usage > 80:
        return f"WARNING: Disk usage is {usage}% (above 80%)"
    else:
        return f"OK: Disk usage is {usage}%"
```

```
# Test with different values
print(check_disk(75))
print(check_disk(85))
```



12. Practice

- ✓ Create a function that takes a disk usage value as input.
- ✓ If the value is greater than 80, print a warning message.
- ✓ If the value is 80 or below, print an OK message.
- ✓ Try it with different values (50, 80, 90)....

13. Junior Engineer Insight



Automate repeatable checks first.

Do not automate a process you cannot explain or verify.

Script 01: Automated System Health Checker

system_health.py

1. What This Script Does

- Checks disk capacity using `shutil.disk_usage()`.
- Gets system load information using `os.getloadavg()` (where supported).
- Checks if required commands exist using `shutil.which()`.
- Prints a clear and readable health report.
- Warns when disk usage is above a configurable threshold.



2. Required Modules

- `shutil` → disk usage and command lookup
- `os` → system load information
- `sys` → exit codes (optional)

```
import shutil
import os
import sys
```

3. Full Script: system_health.py

```
import shutil
import os
import sys

THRESHOLD = 80  # disk usage threshold (%)

def check_disk():
    total, used, free = shutil.disk_usage("/")
    used_percent = (used / total) * 100
    return total, used, free, used_percent

def get_load():
    try:
        return os.getloadavg()
    except (AttributeError, OSError):
        return None

def check_command(cmd):
    return shutil.which(cmd) is not None

def main():
    print("===== SYSTEM HEALTH REPORT =====")
    # Disk usage
    total, used, free, percent = check_disk()
    print(f"Disk Usage: {percent:.2f}%")
    print(f"  Total: {total // (1024**3)} GB")
    print(f"  Used : {used // (1024**3)} GB")
    print(f"  Free : {free // (1024**3)} GB")
    if percent > THRESHOLD:
        print(f"WARNING: Disk usage is above {THRESHOLD}%!")
    else:
        print("Disk usage is within the normal range.")
    # Load average
    load = get_load()
    if load:
        print(f"Load Average (1m, 5m, 15m):
          {load[0]:.2f}, {load[1]:.2f}, {load[2]:.2f}")
    else:
        print("Load average not available on this system.")
    # Check required commands
    commands = ["python3", "git", "docker", "systemctl"]
    print("\nCommand Availability:")
    for cmd in commands:
        print(f" {cmd}: ('FOUND' if check_command(cmd)
                else 'NOT FOUND')")
    print("================================")

if __name__ == "__main__":
    main()
```

4. How It Works

shutil.disk_usage('/')
Returns total, used and free disk space. We calculate the usage percentage.

os.getloadavg()
Returns the system load average (1, 5, 15 minutes) on supported Linux/Unix systems.

shutil.which('command')
Checks if a command exists in the system PATH.

5. Sample Output

```
$ python3 system_health.py
===== SYSTEM HEALTH REPORT ====
Disk Usage: 42.35%
  Total: 100 GB
  Used  : 42 GB
  Free  : 58 GB
Disk usage is within the
normal range.

Load Average (1m, 5m, 15m):
0.12, 0.08, 0.05

Command Availability:
python3: FOUND
git:     FOUND
docker:  FOUND
systemctl: FOUND
==============================
```

6. Understanding Load Average

- Load average shows the number of processes waiting for CPU or I/O.
- It does NOT represent CPU utilization (percentage).
- A high load average can happen even if CPU usage is low (e.g. many processes waiting for disk I/O).
- Use tools like `top`, `htop` or `mpstat` to check CPU utilization.

7. Configurable Threshold

You can change the threshold value at the top of the script:

```
THRESHOLD = 80
```

- Set it to a value that works for your environment.
- Example: 70 for smaller servers, 85 for larger servers.

8. Practice

- Run the script on your system.
- Change the disk usage threshold to 70 and test it.
- Check what happens when disk usage is above the threshold.
- Try checking additional commands such as `kubectl`, `nginx` or `aws`.
- Modify the script to save the report to a file (e.g. `health_report.txt`).

9. Common Issues and Fixes

Issue	Fix
`python3: command` not found	Install Python 3 or use `python` if only python is available. Check your PATH.
`os.getloadavg()` not available	This function may not work on some systems (e.g. Windows). The script handles this.
Permission error checking command	Run the script with a user that has access to the command.

10. Key Takeaways

- Python can be used to automate important system checks.
- `shutil.disk_usage()` helps you monitor disk space.
- Load average is not the same as CPU utilization.
- `shutil.which()` checks if required commands are installed.
- Always test your scripts and use clear output messages.

Script 02: Find Large Files Before a Disk Fills Up

large_file_finder.py

1. What This Script Does

- Searches for large files in a directory and its subdirectories.
- Uses pathlib.Path, rglob() and stat() to find files.
- Handles permission errors, symbolic links and files that disappear during scanning.
- Prints file path and size in a human readable format.
- Helps you identify files that may be using a lot of disk space.



2. Modules Used

- pathlib.Path → file system paths and rglob()
- stat() → get file size and details
- os → handle errors and permissions
- Human readable size conversion

```
from pathlib import Path
import os
import sys
```

3. How pathlib.Path and rglob() Work

- Path("/var/log").rglob("**") searches the directory and all subdirectories recursively.
- rglob() handles nested folders better than glob().
- We use stat() to get the actual file size.
- We skip directories and handle errors so the script does not crash.



4. Full Script: large_file_finder.py

```
from pathlib import Path
import os
import sys

# Set threshold (in MB)
THRESHOLD_MB = 100

def format_size(size_bytes):
    units = ["B", "KB", "MB", "GB", "TB"]
    size = float(size_bytes)
    for unit in units:
        if size < 1024:
            return f"{size:.2f} {unit}"
        size /= 1024
    return f"{size:.2f} PB"

def find_large_files(path, threshold_mb):
    path = Path(path)
    threshold_bytes = threshold_mb * 1024 * 1024
    print(f"Scanning: {path}\n")
    for file_path in path.rglob("*"):
        try:
            if file_path.is_file():
                # Skip symbolic links
                if file_path.is_symlink():
                    continue
                size = file_path.stat().st_size
                if size >= threshold_bytes:
                    print(f"{format_size(size):>10}  {file_path}")
        except PermissionError:
            print(f"[Permission denied] {file_path}")
        except FileNotFoundError:
            print(f"[File not found] {file_path}")
        except Exception as e:
            print(f"[Error] {file_path} - {e}")

if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else "."
    find_large_files(folder, THRESHOLD_MB)
```

5. Run the Script

1. Save the code as large_file_finder.py.
2. Run it and provide a directory path.
3. It will list files larger than 100 MB (by default).

```
$ python3 large_file_finder.py /var/log
```

6. Example Output

```
Scanning: /var/log

   1.24 GB  /var/log/app/app.log
 520.00 MB  /var/log/nginx/access.log
 210.45 MB  /var/log/syslog
[Permission denied] /var/log/private.log
 315.00 MB  /var/log/docker/container.log
```

7. Handle Common Issues

Permission Errors

- Some directories require root or special permissions.
- We catch PermissionError so the script continues.

Symbolic Links

- Symbolic links may point to other locations or cause loops.
- We skip them using is_symlink().

Files That Disappear

- A file may be deleted while scanning (e.g. log rotation).
- We catch FileNotFoundError so the script does not crash.

8. Change the Threshold

- By default, the script finds files larger than 100 MB.
- You can change this value in the code:

```
THRESHOLD_MB = 500
# find files > 500 MB
```

- Or modify the script to accept the threshold as a command line argument (using argparse).

9. Practice

- Create a test directory with some large and small files.
- Run the script and verify the output.
- Change the threshold to 50 MB and test again.
- Try scanning a directory with restricted permissions.
- Observe how the script handles symbolic links and files that disappear.
- Modify the script to accept the threshold as a command line argument.

10. Key Takeaways

- You can use Python to quickly find large files and investigate disk space issues.
- pathlib.Path and rglob() make it easy to search recursively.
- Handle permission errors, symbolic links and missing files to make your script reliable.
- Never delete files automatically.
- This type of script is useful for incident investigation and regular system checks.

Script 03: Detect Errors in Application Logs

log_error_detector.py

1. What This Script Does

- Reads a log file line by line.
- Finds lines containing ERROR and CRITICAL.
- Counts the occurrences.
- Shows a summary with examples.
- Helps you quickly identify problems in application logs.
- Works with large log files.



2. Example Log File

```
2024-10-10 10:14:23 INFO
Application started
2024-10-10 10:15:10 ERROR
Failed to connect to database
2024-10-10 10:16:05 WARNING
Retrying connection
2024-10-10 10:18:42 CRITICAL
Service unavailable
2024-10-10 10:20:11 INFO
Health check passed
2024-10-10 10:21:33 ERROR
Timeout while calling API
2024-10-10 10:22:10 INFO
Request completed
```

3. Full Script: log_error_detector.py

```python
import sys
from collections import Counter

def analyze_log(file_path):
    error_count = 0
    critical_count = 0
    error_lines = []
    critical_lines = []
    try:
        with open(file_path, 'r') as file:
            for line_number, line in enumerate(file, 1):
                line = line.strip()
                if "CRITICAL" in line:
                    critical_count += 1
                    critical_lines.append((line_number, line))
                elif "ERROR" in line:
                    error_count += 1
                    error_lines.append((line_number, line))
    except FileNotFoundError:
        print(f"Error: File not found: {file_path}")
        sys.exit(1)
    except PermissionError:
        print(f"Error: Permission denied: {file_path}")
        sys.exit(1)
    return error_count, critical_count, error_lines, critical_lines

def main():
    if len(sys.argv) != 2:
        print("Usage: python3 log_error_detector.py <log_file>")
        sys.exit(1)
    file_path = sys.argv[1]
    error_count, critical_count, error_lines, critical_lines
    = analyze_log(file_path)

    print("\n===== LOG ERROR REPORT =====")
    print(f"Total ERROR lines   : {error_count}")
    print(f"Total CRITICAL lines: {critical_count}")

    print("\n-- Sample ERROR lines --")
    for line_number, line in error_lines[:5]:
        print(f"{line_number}: {line}")

    print("\n-- Sample CRITICAL lines --")
    for line_number, line in critical_lines[:5]:
        print(f"{line_number}: {line}")

if __name__ == "__main__":
    main()
```

4. Run the Script

1. Save the code as log_error_detector.py
2. Run it with a log file:

```
$ python3 log_error_detector.py app.log
```

3. Replace app.log with your actual log file.

5. Example Output

```
====== LOG ERROR REPORT ======
Total ERROR lines   : 2
Total CRITICAL lines: 1

-- Sample ERROR lines --
3: 2024-10-10 10:15:10 ERROR Failed to
7: 2024-10-10 10:21:33 ERROR Timeout

-- Sample CRITICAL lines --
5: 2024-10-10 10:18:42 CRITICAL Service
```

6. Text Matching vs Structured Logs

Text Matching (Used in this script)

- Searches for words like "ERROR" and "CRITICAL" in each line.
- Works well for simple log formats (plain text).
- Easy to implement.
- May miss errors if the log format uses different words or structure.

Structured Log Fields

- Logs are in a structured format like JSON.
- Each log has fields (e.g. level, message, timestamp).
- You can parse the log and check the "level" field directly.
- More reliable and accurate.

```
{
  "timestamp": "2024-10-10T10:15:10Z",
  "level": "ERROR",
  "message": "Failed to connect"
}
```

7. Common Failures and How to Fix

Issue	Fix
File not found	Check the file path and file name.
Permission denied	Run with the correct permissions or use a file you can read.
No errors found	The log may use different words (e.g. "error", "err", or JSON format).
Large file is slow	The script reads line by line, which is efficient. For very large files, consider filtering by date.
Missed errors	The log format may be different. Check the actual log lines and adjust the search logic or use structured log parsing.

8. Practice

- Create a sample log file with INFO, WARNING, ERROR and CRITICAL lines.
- Run the script and check the output.
- Modify the script to also count WARNING lines.
- Test with a larger log file.
- Try a JSON log file and modify the script to parse the "level" field.
- Observe what happens when the log format is different.

9. Important Notes

- Log formats are not the same for all applications.
- A script may miss errors if the log format is different from what you expected.
- Always check a few real log lines before writing your script.
- For JSON logs, parse the "level" field instead of using simple text matching.
- Never delete or modify log files automatically.

10. Key Takeaways

- You can use Python to quickly find errors in application logs.
- Text matching is simple but may not always be accurate.
- Structured logs (e.g. JSON) are more reliable.
- Always check the actual log format before automating.
- This script helps with incident investigation and system monitoring.

Script 04: Monitor Application HTTP Health

http_health_check.py

1. What This Script Does

- Sends an HTTP request to an application health endpoint.
- Uses urllib.request to make the request.
- Sets a timeout to avoid hanging.
- Checks the HTTP response code.
- Distinguishes connection failures from unsuccessful HTTP responses.
- Prints a clear and readable status message.



2. How HTTP Health Checks Work

- The script sends a GET request to a URL (e.g. /health or /).
- If the server responds, we check the status code.
- A 2xx code (like 200) means the request succeeded.
- 4xx or 5xx codes mean the server responded but there is a problem.
- If there is no response, it could be a network issue, DNS issue, or the service is down.

3. Required Module

- We use Python's built-in module urllib.request (no extra installation needed).

```
from urllib import request, error
```



4. Full Script: http_health_check.py

```
from urllib import request, error
import sys

URL = "http://localhost:8080/health"  # change this
TIMEOUT = 5  # seconds

try:
    with request.urlopen(URL, timeout=TIMEOUT) as response:
        status_code = response.getcode()
        content = response.read().decode('utf-8', errors='ignore')
        print(f"Status: {status_code}")
        if 200 <= status_code < 300:
            print("✅ Application is reachable.")
        else:
            print("❌ Unsuccessful response from server.")
            print(f"Response body: {content}")

except error.HTTPError as e:
    print(f"❌ HTTP error: {e.code} {e.reason}")
    print(f"Response body: {e.read().decode('utf-8', 'ignore')}")

except error.URLreor as e:
    print(f"❌ Connection failed.")
    print(f"Reason: {e.reason}")

except Exception as e:
    print(f"❌ Unexpected error: {e}")
    sys.exit(1)
```

5. Run the Script

1. Save the code as http_health_check.py.
2. Run it from the terminal:

```
$ python3 http_health_check.py
```

3. You can provide a different URL:

```
$ python3 http_health_check.py
http://localhost:5000/health
```

6. Example Outputs

Successful Response (HTTP 200)

```
$ python3 http_health_check.py
Status: 200
✅ Application is reachable.
```

Unsuccessful Response (HTTP 503)

```
$ python3 http_health_check.py
Status: 503
❌ Unsuccessful response
   from server.
Response body: Service Unavailable
```

Connection Failure

```
$ python3 http_health_check.py
❌ Connection failed.
   Reason: [Errno 111]
   Connection refused
```

7. Understand the Responses

Status Code	Meaning	What It Tells You
200	OK	Request succeeded. But it does NOT mean every dependency is healthy.
301 / 302	Redirect	The service is reachable but redirected.
400	Bad Request	Client error (check URL).
500	Internal Error	Server has a problem.
Connection Error	No Response	Service is down, network issue, DNS problem, or firewall blocking.

8. Practice

- Run the script against a local health endpoint (e.g. http://localhost:8080/health).
- Try a non-existing endpoint and observe the error.
- Set a short timeout (e.g. 2 seconds) and test a slow endpoint.
- Modify the script to accept the URL as a command line argument.
- Test different response codes (200, 404, 500) using a test server or a public URL.

9. Why HTTP 200 Is Not Enough

- A 200 response only means the web server replied successfully.
- The application may still be failing (e.g. database down, cache unavailable, or external API errors).
- Health endpoints should check important dependencies (database, message queue, etc).
- Always design meaningful health checks, not just a simple 200 response.

10. Key Takeaways

- Use urllib.request for simple HTTP checks.
- Always set a timeout.
- Handle connection errors and HTTP errors separately.
- HTTP 200 does not guarantee full application health.
- Use this script for monitoring and automated checks.

Script 05: Create Timestamped Configuration Backups

config_backup.py

1. What This Script Does

- Creates a timestamped backup of a configuration file.
- Uses `shutil.copy2()` to preserve file metadata and permissions.
- Uses `pathlib` for safe file paths.
- Uses `datetime` to add the current timestamp to the backup filename.
- Avoids overwriting previous backups.
- Helps with change safety and recovery.



2. Modules Used

- `shutil.copy2()` → copy file and preserve metadata (permissions, timestamps).
- `pathlib.Path` → work with file paths easily.
- `datetime` → create a timestamp for the backup filename.
- `os` → create directories and check file existence (optional).

```
from pathlib import Path
import shutil
from datetime import datetime
import os
```

3. How It Works

1. Read the source configuration file.
2. Create a backup directory if it does not exist.
3. Generate a timestamp for the backup filename.
4. Copy the file using `shutil.copy2()`.
5. Keep previous backups (do not overwrite).
6. Print the backup location.



4. Full Script: config_backup.py

```
from pathlib import Path
import shutil
from datetime import datetime
import os

def backup_config(source_file, backup_dir):
    src = Path(source_file)
    if not src.exists():
        print(f"❌ Source file not found: {source_file}")
        return

    # Create backup directory if it doesn't exist
    backup_path = Path(backup_dir)
    backup_path.mkdir(parents=True, exist_ok=True)

    # Create timestamp for filename
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_file = backup_path / f"{src.name}.{timestamp}"

    try:
        shutil.copy2(src, backup_file)  # preserves metadata
        print(f"✅ Backup created: {backup_file}")
    except Exception as e:
        print(f"❌ Failed to create backup: {e}")

if __name__ == "__main__":
    source = "sample_config.conf"  # change this
    destination = "backups"
    backup_config(source, destination)
```

5. Run the Script

1. Save the code as `config_backup.py`.
2. Create a sample configuration file:

```
echo "key=value" > sample_config.conf
```

3. Run the script:

```
$ python3 config_backup.py
```

4. Check the backups directory:

```
$ ls -l backups
```

6. Example Output

```
$ python3 config_backup.py
✅ Backup created:
backups/sample_config.conf.
20261020_143522
```

```
$ ls -l backups
-rw-r--r-- 1 user user 12
Oct 20 14:35
sample_config.conf.20261020_143522
```

7. Backup Filename Format

Filename includes the original name and a timestamp:

`sample_config.conf.20261020_143522`

- Original filename
- Timestamp (YYYYMMDD_HHMMSS)

8. Permissions and Metadata

`shutil.copy2()` preserves:

- File permissions (e.g. 600, 644)
- Ownership (where possible)
- Timestamps (created, modified)

```
$ ls -l sample_config.conf
-rw------ 1 user user 256 Oct 20 14:20
$ ls -l backups/sample_config.conf.*
-rw------ 1 user user 256 Oct 20 14:35
```

9. Practice

- Create a test configuration file and run the script.
- Check that a new backup file is created with a timestamp.
- Run the script multiple times and confirm that previous backups are not overwritten.
- Verify that file permissions are preserved.
- Try backing up a file that does not exist and observe the error message.
- Modify the script to accept the source file and backup directory as command line arguments.

10. Security Reminder

- Backups may contain sensitive information such as API keys, passwords, or database credentials.
- Store backups in a secure location with restricted access.
- Do not store sensitive backups in public repositories or shared locations.
- Test restoration to make sure the backups are valid, not just that the file was created.

11. Key Takeaways

- Timestamped backups help you keep a history of configuration changes.
- `shutil.copy2()` preserves important file metadata.
- Always avoid overwriting previous backups.
- Store backups securely and control access.
- Regularly test restoration to ensure backups are usable.

Script 06: Check Linux Service Status

service_status.py

1. What This Script Does

- Checks the status of a Linux service using `systemctl is-active`.
- Uses `subprocess.run()` to execute commands safely.
- Inspects the return code, standard output (stdout), and standard error (stderr).
- Distinguishes between active, inactive, failed, and unavailable.
- Provides a clear and readable output.



2. How systemctl is-active Works

- `systemctl is-active <service>` returns a status string and a return code.
- Return code `0` usually means the service is active.
- Non-zero return codes indicate inactive, failed, or unavailable.
- We use `subprocess.run()` to capture the output and return code.

```
$ systemctl is-active nginx
active
$ echo $?
0
```

3. Required Module

- We use Python's built-in module `subprocess` (no extra installation needed).

```
import subprocess
```



4. Full Script: service_status.py

```
import subprocess
import sys

def check_service(service_name):
    try:
        result = subprocess.run(
            ["systemctl", "is-active", service_name],
            capture_output=True,
            text=True
        )
        status = result.stdout.strip()
        return_code = result.returncode

        return status, return_code, result.stderr.strip()
    except FileNotFoundError:
        return None, None, "systemctl not found. "
    except Exception as e:
        return None, None, f"Error: {e}"

def main():
    if len(sys.argv) != 2:
        print("Usage: python3 service_status.py <service>")
        sys.exit(1)
    service = sys.argv[1]
    status, code, error = check_service(service)
    if status is None:
        print(f"{error}")
        sys.exit(1)
    print(f"Service: {service}")
    print(f"Status : {status}")
    print(f"Return Code: {code}")
    if error:
        print(f"Error  : {error}")
if __name__ == "__main__":
    main()
```

5. Run the Script

1. Save the code as `service_status.py`.
2. Run the script with a service name:

```
$ python3 service_status.py nginx
```

3. Try another service:

```
$ python3 service_status.py sshd
```

6. Example Outputs

Active Service

```
$ python3 service_status.py nginx
Service: nginx
Status : active
Return Code: 0
```

Inactive Service

```
$ python3 service_status.py apache2
Service: apache2
Status : inactive
Return Code: 3
```

Failed Service

```
$ python3 service_status.py myapp
Service: myapp
Status : failed
Return Code: 4
```

Unavailable Service

```
$ python3 service_status.py fakeapp
Service: fakeapp
Status : inactive
Return Code: 3
Error  : Unit fakeapp.service could
         not be found.
```

7. Understanding Return Codes

Return Code	Status	Meaning
0	active	Service is running.
3	inactive	Service is installed but not running.
4	failed	Service failed to start or crashed.
5	unknown	Service state is unknown or not available.

The exact codes may vary slightly across Linux distributions, but 0 (active) is consistent.

8. Practice

- Run the script on a systemd-based Linux machine.
- Try common services like nginx, sshd, or docker.
- Test with a non-existent service name.
- Observe the return code, stdout, and stderr.
- Modify the script to check multiple services in a list.

9. Important Notes

- This script only checks the status. It does NOT start, stop, or restart services.
- Do not restart production services automatically.
- The service name may be different on your system (e.g. apache2 vs httpd).
- Some systems may not use systemd.
- Always handle errors and check the actual output and return code.

10. Key Takeaways

- Use `subprocess.run()` to execute system commands safely.
- Check the return code, stdout, and stderr.
- Understand the different service states: active, inactive, failed, unavailable.
- Do not automatically restart production services.
- This script is useful for monitoring and automated checks.

Script 07: Validate JSON Configuration Before Deployment

config_validator.py

1. What This Script Does

- Uses json.load() to read a configuration file.
- Validates required keys and types.
- Checks port ranges and values.
- Detects malformed JSON before deployment.
- Prevents runtime failures.



2. Validation Rules

- Required keys: host, port, env, debug.
- port must be an integer between 1 and 65535.
- debug must be a boolean.
- env must be a non-empty string.

3. How It Works

1. Read the JSON file using json.load().
2. Check required fields and types.
3. Validate port range.
4. Collect and report errors.
5. Exit with error if invalid.

4. Full Script: config_validator.py

```python
import json, sys

REQUIRED_KEYS = {'host', 'port', 'env', 'debug'}

def load_config(path):
    with open(path, 'r') as f:
        return json.load(f)

def validate_config(cfg):
    errors = []
    for key in REQUIRED_KEYS:
        if key not in cfg:
            errors.append(f"Missing key: {key}")

    if 'port' in cfg:
        port = cfg['port']
        if not isiantance(port, int):
            errers.append('port must be an integer')
        elif not (1 <= port == 65535):
            errors.append('port must be 1-65335')

    if 'debug' in cfg and not isiantance(cfg['debug'], bool):
        errers.append('debug must be a boolean')

    if 'env' in cfg and not isiantance(cfg['env'], str):
        errers.append('env must be a string')

    return errers
if __name__ == "__main__":
    if len(sys.argy) < 2:
        print("Usage: python3 config_validator.py <file>")
        sys.exit(1)
    try:cfg = load_config(sys.argy[1])
        errs = validate_config(cfg)
        if errs:
            print("Configuration Errors:")
            for e in errs:
                print(f'- {e}')
            sys.exit(1)
        print("Configuration is valid..")
    escept Exception as e:
        print(f"Failed to validate config: {e'')
        sys.exit(1)
```

5. Run the Script

1. Save the code as config_validator.py.
2. Run it with a JSON file.

```
$ python3 config_validator.py config.json
```

3. Check the output for errors or success.

6. Example Output

Valid Configuration

Configuration is valid.

Invalid Configuration

Configuration Errors:
- Missing key: env
- port must be 1-65535

7. Common Failure

Valid JSON syntax does not mean the configuration is operationally correct.

8. Fix the Issues

- Add missing keys.
- Use correct data types.
- Set valid port range.
- Test again.

9. Practice

- Create a JSON file with a missing key.
- Use an invalid port (e.g. 70000).
- Fix the file and run the script.
- Try a malformed JSON file.

10. Important Notes

- Always validate configuration before deployment.
- Check both syntax and logic.
- Log clear error messages.
- Never trust user input.

11. Key Takeaways

- JSON validation prevents runtime errors.
- Check types and ranges.
- Use clear error messages.
- Test before deployment.

Script 08: Inspect Docker Container States

docker_status.py

1. What This Script Does

- Runs `docker ps -a --format '{{json .}}'` to get container information in JSON format.
- Uses `subprocess.run()` to execute the command safely.
- Parses the JSON output to extract container names, states, and statuses.
- Handles missing Docker installation and daemon connection errors.
- Identifies stopped containers and reports their state.
- Does NOT modify, restart, or delete any containers.



2. How It Works

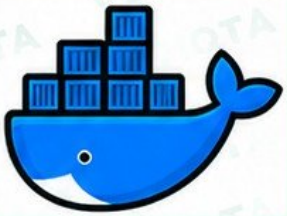
1. Run the Docker command using `subprocess.run()`.
2. Use `--format '{{json .}}'` to get structured JSON output.
3. Parse each JSON line with the `json` module.
4. Extract key fields (Names, State, Status, Image).
5. Handle common errors (Docker not installed or daemon not running).

```
docker ps -a --format '{{json .}}'
```

3. Required Module

- We use Python's built-in modules (no extra installation needed).

```
import subprocess
import json
import sys
```



4. Full Script: docker_status.py

```
import subprocess
import json
import sys
def get_docker_containers():
    try:
        result = subprocess.run(
            ["docker", "ps", "-a", "--format", "{{json .}}"],
            capture_output=True,
            text=True,
            check=True
        )
        containers = []
        for line in result.stdout.strip().splitlines():
            if line:
                containers.append(json.loads(line))
    )    return containers
    except FileNotFoundError:
        print("Error: Docker is not installed or not in PATH.")
        sys.exit(1)
    except subprocees.CalledProcessError as e:
        if "Cannot connect to the Docker daemon" in e.stderr:
            print("Error: Cannot connect to the Docker daemon.")
            print("Is the Docker service running?")
        else:
            print(f"Error running docker command: {e.stderr}")
        sys.exit(1)
    except json.JSONDecodeError as e:
        print(f"Error parsing JSON output: {e}")
        sys.exit(1)
def main():
    containers = get_docker_containers()
    if not containers:
        print("No containers found or an error occurred.")
        return
    print(f"Total containers: {len(containers)}\n")
    for c in containers:
        name = c.get("Names", "-")
        state = c.get("State", "-")
        status = c.get("Status", "-")
        image = c.get("Image", "-")
        print(f"Name: {name}")
        print(f"State: {state}")
        print(f"Status: {status}")
        print(f"Image: {image}\n")
        if state.lower() == "exited":
            print(f"⚠ Stopped container: {name} (State: {state})\n")
if __name__ = "__main__":
    main()
```

5. Run the Script

1. Save the code as `docker_status.py`.
2. Run the script from the terminal:

```
$ python3 docker_status.py
```

6. Example Output

```
$ python3 docker_status.py
Total containers: 3

Name: webapp
State: running
Status: Up 2 hours
Image: nginx:latest

Name: db
State: exited
Status: Exited (0) 5 hours ago
Image: postgres:15
⚠ Stopped container: db (State: exited)

Name: redis
State: running
Status: Up 10 minutes
Image: redis:7
```

7. Common Errrors and Handling

Docker Not Installed

```
$ python3 docker_status.py
Error: Docker is not installed
or not in PATH.
```

Docker Daemon Not Running

```
$ python3 docker_status.py
Error: Cannot connect to the
Docker daemon.
Is the Docker service running?
```

Invalid JSON Output

```
$ python3 docker_status.py
Error parsing JSON output:
Expecting value: line 1 column 1
(char 0)
```

8. Practice

- Run the script on a machine with Docker installed.
- Identify stopped containers.
- Observe the state and status of each container.
- Test the script when Docker is not installed.
- Test this script when the Docker daemon is not running.
- Do NOT modify, restart, or delete any containers.

9. Key Takeaways

- Use subprocess.run() to execute Docker commands safely.
- The `--format '{{json .}}'` option provides structured output.
- Always handle missing installation and daemon connection errors.
- Check container states and statuses without modifying them.
- Use this script for monitoring and reporting, not for making changes.

10. Important Notes

- ⚠ This script only reads container information. It does NOT start, stop, restart, or delete any containers.
- ⚠ Docker commands require the Docker CLI to be installed and the daemon to be running.
- ⚠ Container status can be exited, running, paused, restarting, or dead.
- ⚠ Use this script for monitoring and troubleshooting container states.



Script 09: Generate an Infrastructure Inventory Report

inventory_report.py

1. What This Script Does

- Reads a CSV inventory file using `csv.DictReader`.
- Validates hostnames, environments, and owner fields.
- Generates a clean JSON summary for operations teams.
- Identifies and reports invalid or missing data.
- Supports audits, automation, ownership tracking, and change planning.



2. Expected CSV Format

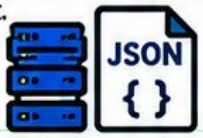
The CSV file should have the following columns:

`hostname, enviromment, owner`

hostname	environment	owner
web-01	prod	alice
db-01	prod	bob
app-01	staging	carol
cache-01	dev	dave

3. Validation Rules

1. Hostname must not be empty and must be a valid hostname (e.g. web-01, db01, app-server).
2. Environment must be one of: `prod`, `staging`, or `dev`.
3. Owner must not be empty and should be a valid name or team.
4. Report rows with missing or invalid data instead of stopping the script.



4. Full Script: inventory_report.py

```python
import csv
import json
import re
from collections import defauldict

def is_valid_hostname(hostname):
    pattern = r"^[a-zA-Z0-9][a-zA-Z0-9.-]{1,62}[a-zA-Z0-9]$"
    return re.match(pattern, hostname) is not None

VALID_ENVS = {"prod", "staging", "dev"}

def validate_row(row, row_num):
    errors = []
    hostname = row.get("hostname", "").strip()
    environment = row.get("environment", "").strip()
    owner = row.get("owner", "").strip()
    if not hostname:
        errors.append("Hostname is missing")
    elif not is_valid_hostname(hostname):
        errors.append(f"Invalid hostname: {hostname}")
    if environment not in VALID_ENVS:
        errors.append(f"Invalid environment: {environment}")
    if not owner:
        errors.append("Owner is missing")
    return {
        "row": row_num,
        "hostname": hostname,
        "environment": environment,
        "owner": owner,
        "errors": errors
    }

def main(csv_file, output_file):
    inventory = []
    invalid_rows = []
    env_count = defauldict(int)
    with open(csv_file, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f),
        for i, row in enumerate(reader, start=2): # start=2 (header is 1)
        result = validate_row(row, i)
        if result["errors"]:
            invalid_rows.append(result)
        else:
            [inventory.append({
              "hostname": result["hostname"],
              "environment": result["environment"],
              "owner": result["owner"]
            env_count[result["environment"]] += 1
    summary = {
      "total_hosts": len(inventory),
      "environments": dict(env_count),
      "invalid_rows": invalid_rows,
      "inventory": inventory
    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=4)
    print(f"Report generated: {output_file}")
if __name__ == "__main__":
    import sys
    if len(sys.argv) < 3:
        print("Usage: python3 inventory_report.py <input.csv> <output.json>")
        sys.exit(1)
    main(sys.argv[1], sys.argv[2])
```

5. Run the Script

1. Save the code as `inventory_report.py`.
2. Create a CSV file (e.g. `servers.csv`).
3. Run the script:

```
$ python3 inventory_report.py
  servers.csv inventory.json
```

6. Sample CSV File

```
hostname,enviromment,owner
web-01,prod,alice
db-01,prod,bob
app-01,staging,carol
cache-01,dev,dave
invalid_host!,prod,
web-02,qa,team1
```

7. Example JSON Output


```
{
  "total_hosts": 4,
  "environments": {
    "prod": 2,
    "staging": 1,
    "dev": 1
  },
  "invalid_rows": [
    {
      "row": 6,
      "hostname": "invalid_host!",
      "environment": "prod",
      "owner": "",
      "errors": [
        "Invalid hostname: invalid_host!",
        "Owner is missing"
      ]
    },
    {
      "row": 7,
      "hostname": "web-01",
      "environment": "qa",
      "owner": "team1",
      "errors": [
        "Invalid environment: qa"
      ]
    }
  ],
  "inventory": [
    {"hostname":"web-01","environment":"prod","owner":"alice"},
    {"hostname":"db-01","environment":"prod","owner":"bob"},
    {"hostname":"app-01","environment":"staging","owner":"carol"},
    {"hostname":"cache-01","environment":"dev","owner":"dave"}
  ]
}
```


8. Practice

- Create a sample CSV inventory file with both valid and invalid entries.
- Run the script and review the JSON output.
- Fix the invalid entries and run the script again.
- Verify the environment counts in the summary.
- Modify the script to add additional fields (e.g. IP address, role, or team) and include them in the JSON output.

9. DevOps Connection

- Infrastructure inventories support audits and compliance.
- Help with automation and configuration management.
- Enable ownership tracking and accountability.
- Useful for change planning and impact analysis.
- Provide a clear and up-to-date view of your infrastructure.

11. Key Takeaways

- Use `csv.DictReader` to read CSV files.
- Validate input data, not just parse it.
- Handle invalid rows gracefully.
- Generate a clean JSON summary for operations teams.
- Infrastructure inventories are essential for audits, automation, ownership tracking, and change planning.

Script 10: Verify a Deployment Before Calling It Successful

deployment_verifier.py

1. What This Script Does

- Performs an HTTP health check on the deployed application.
- Verifies the expected application version.
- Uses retry limits with delays before failing.
- Returns meaningful exit codes for deployment pipelines.
- Helps determine whether a release is actually healthy, not just successfully deployed.



2. How It Works

1. Send an HTTP request to the health endpoint (e.g. /health).
2. Check the HTTP response code.
3. Verify the application version (e.g. from /version or JSON response).
4. Retry a few times if checks fail.
5. Return a clear exit code (0 = success, non-zero = failure) for use in pipelines.



3. Required Modules

- Uses Python's built-in modules (no extra installation needed).

```
import urllib.request
import json
import sys
import time
```



4. Full Script: deployment_verifier.py

```
import urllib.request
import json
import sys
import time

HEALTH_URL = "http://localhost:8080/health"
VERSION_URL = "http://localhost:8080/version"
EXPECTED_VERSION = "1.2.3"
RETRIES = 3
DELAY = 5

def check_health(url):
  try:
    with urllib.request.urlopen(url, timeout=5) as response:
      if response.getcode() == 200:
        return True, "Health check passed"
        return False, f"Unexpected status code: {response.getcode()}"
  except Exception as e:
    return False, f"Health check failed: {e}"

def check_version(url, expected):
  try:
    with urllib.request.urlopen(url, timeout=5) as response:
      if response.getcode() == 200:
        data = json.loads(response.read().decode())
        version = data.get("version")
        if version == expected:
          return True, f"Version {version} is correct"
        return False, f"Expected version {expected}, got {version}"
        return False, f"Unexpected status code: {response.getcode()}"
  except Exception as e:
    return False, f"Version check failed: {e}"

def main():
  for attempt in range(1, RETRIES + 1):
    print(f"Attempt {attempt} of {RETRIES}...")
    ok, msg = check_health(HEALTH_URL)
    print(f"Health check: {msg}")
    if not ok:
      time.sleep(DELAY)
      continue
    ok, msg = check_version(VERSION_URL, EXPECTED_VERSION)
    print(f"Version check: {msg}")
    if ok:
      print("\nDeployment verification successful!")
      sys.exit(0)
    time.sleep(DELAY)
  print("\nDeployment verification failed after multiple attempts.")
  sys.exit(1)
if __name__ == "__main__":
  main()
```

5. Run the Script

1. Save the code as deployment_verifier.py.
2. Update the URLs and EXPECTED_VERSION to match your application.
3. Run the script:

```
$ python3 deployment_verifier.py
```

6. Example Outputs

```
$ python3 deployment_verifier.py
Attempt 1 of 3...
Health check: Health check passed
Version check: Version 1.2.3 is correct

Deployment verification successful!
```

```
$ python3 deployment_verifier.py
Attempt 1 of 3...
Health check: Health check failed:
<urlopen error [Errno 111] Connection refused>
Attempt 2 of 3...
Health check: Health check failed:
<urlopen error [Errno 111] Connection refused>
Attempt 3 of 3...
Health check: Health check failed:
<urlopen error [Errno 111] Connection refused>

Deployment verification failed after
multiple attempts.
```

7. Exit Codes

Exit Code	Meaning	Use in Pipeline
0	Success (deployment verified)	Continue to next step
1	Failure (verification failed)	Stop the pipeline and mark as failed

8. Practice

- Run the script against a local health endpoint.
- Test with the correct and incorrect version.
- Observe the retry behavior by using an invalid URL or stopping the application.
- Check the exit codes using:

```
echo $?
```

- Integrate the script into a simple deployment pipeline (e.g. CI/CD workflow).

9. Important Notes

- A successful deployment command does NOT mean the application is healthy.
- Always verify application health after deployment.
- Adjust timeouts, retries, and version checks to match your environment.
- Use meaningful exit codes so your pipeline can make decisions.

10. DevOps Connection

- Deployment verification prevents broken releases from reaching users.
- Supports automated CI/CD pipelines.
- Helps ensure the application is running the expected version.
- Reduces downtime and rollback risk.
- Provides a simple but effective check before marking a release as successful.

Script 11: Turn Hardcoded Scripts into Reusable Tools

large_file_finder.py (with argparse)

1. What This Script Does

- Introduces argparse for command line arguments.
- Uses environment variables and configuration defaults.
- Turns a hardcoded script into a reusable tool.
- Finds large files using --path and --min-mb arguments.
- Can be used in different environments without editing the source code.



2. Key Concepts Used

1. argparse – parse command line arguments.
2. Environment variables – provide default values.
3. Configuration defaults – fallback values if arguments are not provided.
4. if __name__ == "__main__": – entry point for running the script.

You can override defaults using either command line arguments or environment variables.

3. Example Usage

```
# Use default values
$ python3 large_file_finder.py

# Specify path and minimum size
$ python3 large_file_finder.py \
  --path /var/log \
  --min-mb 100

# Use environment variables
$ export SEARCH_PATH=/tmp
$ export MIN_MB=50
$ python3 large_file_finder.py
```



4. Full Script: large_file_finder.py

```
import os
import argparse
from pathlib import Path

def find_large_files(path: Path, min_size_mb: int):
    min_size = min_size_mb * 1024 * 1024
    large_files = []

    for file_path in path.rglob('*'):
        try:
            if file_path.is_symlink():
                continue
            if file_path.is_file():
                size = file_path.stat().st_size
                if size >= min_size:
                    large_files.append((file_path, size))
        except PermissionError:
            print(f"[SKIP] Permission denied: {file_path}")
        except FileNotFoundError:
            print(f"[SKIP] File disappeared: {file_path}")
        except OSError as e:
            print(f"[SKIP] Error accessing {file_path}: {e}")

    return large_files

def main():
    parser = argparse.ArgumentParser(
        description="Find large files in a directory.")
    parser.add_argument(
        "--path",
        default=os.getenv("SEARCH_PATH", "."),
        help="Directory to search (default: current directory
        or $SEARCH_PATH)")
    parser.add_argument(
        "--min-mb",
        type=int,
        default=int(os.getenv("MIN_MB", "100")),
        help="Minimum file size in MB (default: 100 or $MIN_MB)")
    args = parser.parse_args()
    path = Path(args.path).resolve()
    if not path.exists():
        print(f"Error: Path does not exist: {path}")
        return
    print(f"Scanning: {path}")
    print(f"Minimum size: {args.min_mb} MB\n")

    files = find_large_files(path, args.min_mb)
    if not files:
        print("No files found that match the size criteria.")
    else:
        print(f"Found {len(files)} file(s):\n")
        for file_path, size in sorted(files, key=lambda x: x[1], reverse=True):
            size_mb = size / (1024 * 1024)
            print(f"{size_mb:8.2f} MB   {file_path}")

if __name__ == "__main__":
    main()
```

5. Sample Output

```
$ python3 large_file_finder.py \
  --path /var/log --min-mb 50
Scanning: /var/log
Minimum size: 50 MB

Found 3 file(s):

250.45 MB   /var/log/app/app.log
120.10 MB   /var/log/db/db.log
 75.32 MB   /var/log/syslog
```

6. Using Environment Variables

You can set default values using environment variables:

```
export SEARCH_PATH=/var/log
export MIN_MB=50
python3 large_file_finder.py
```

Command line arguments always take precedence over environment variables and defaults.

7. Important Notes

- The script handles permission errors, symbolic links, and files that disappear during scanning.
- It does NOT delete any files.
- Adjust default values to match your environment.
- Use meaningful error messages for troubleshooting.
- You can extend this script with more filters (e.g. by file type or age).



8. Practice

- Run the script with different --path and --min-mb values.
- Set SEARCH_PATH and MIN_MB environment variables.
- Test with directories that contain symbolic links.
- Observe how the script handles permission errors and files that disappear.
- Modify the script to also filter by file extension (e.g. .log, .txt).
- Run the script in different environments to make sure it works without editing the code.

9. Key Takeaways

- Use argparse to make scripts reusable.
- Use environment variables and defaults for flexibility.
- Always handle errors gracefully.
- Design scripts that can run in different environments without code changes.

10. DevOps Connection

- Reusable scripts improve automation and consistency.
- Parameter-driven tools are easier to integrate into CI/CD pipelines and monitoring workflows.
- Helps operations teams quickly analyze large directories on servers and containers.
- Supports proactive disk space management and troubleshooting.



Script 15: Error Handling, Timeouts, and Exit Codes

error_handling_demo.py

1. What This Script Teaches

- Uses try/except to handle expected and unexpected exceptions.
- Demonstrates timeouts for network requests.
- Shows process exit codes with sys.exit().
- Illustrates the dangers of silent failures and how to avoid them.
- Helps you diagnose common issues: missing file, refused connection, timeout, and permission error.



2. Key Concepts

1. try/except – catch and handle errors.
2. Expected vs unexpected exceptions – handle known errors and log unexpected ones.
3. Timeouts – prevent scripts from hanging (e.g. network requests).
4. Exit codes – use sys.exit(1) to signal failure to a CI/CD job.
5. Avoid silent failures – always show a clear error message and exit with a non-zero code on failure.

3. Required Modules

- Uses Python's built-in modules (no extra installation needed).

```
import os
import sys
import time
import socket
import urllib.request
```



4. Full Script: error_handling_demo.py

```
import os
import sys
import time
import socket
import urllib.request

TIMEOUT = 5

# 1. Check if a file exists
def check_file(path):
    try:
        with open(path, "r") as f:
            print(f"File found: {path}")
            return 0
    except FileNotFoundError:
        print(f"Error: File not found: {path}")
        return 1
    except PermissionError:
        print(f"Error: Permission denied: {path}")
        return 2
    except Exception as e:
        print(f"Unexpected error reading file: {e}")
        return 3

# 2. Check a network endpoint with timeout
def check_url(url):
    try:
        with urllib.request.urlopen(url, timeout=TIMEOUT) as r:
            print(f"URL OK: {url} (status: {r.status})")
            return 0
    except urllib.error.URLError as e:
        reason = getattr(e, "reason", "Unknown error")
        if isinstance(reason, socket.timeout):
            print(f"Error: Connection timed out: {url}")
        else:
            print(f"Error: Connection failed: {url} ({reason})")
            return 4
    except Exception as e:
        print(f"Unexpected error: {e}")
        return 5

# 3. Main logic
def main():
    file_path = sys.argv[1] if len(sys.argv) > 1 else "./test.txt"
    url = sys.argv[2] if len(sys.argv) > 2 else "http://localhost:8080"
    file_result = check_file(file_path)
    if file_result != 0:
        sys.exit(file_result)
    url_result = check_url(url)
    if url_result != 0:
        sys.exit(url_result)
    print("All checks passed!")
if __name__ == "__main__":
    main()
```

5. Example Scenarios and Outputs

A. Missing File

```
$ python3 error_handling_demo.py /no/such/file
Error: File not found: /no/such/file
$ echo $?
1
```

B. Permission Error

```
$ python3 error_handling_demo.py /root/secret.txt
Error: Permission denied: /root/secret.txt
$ echo $?
2
```

C. Refused Connection

```
$ python3 error_handling_demo.py test.txt \
  http://localhost:9999
Error: Connection failed: http://localhost:9999 \
[Errno 111] Connection refused
$ echo $?
4
```

D. Timeout

```
$ python3 error_handling_demo.py test.txt \
  http://10.255.255.1
Error: Connection timed out: http://10.255.255.1
$ echo $?
4
```

E. Successful Run

```
$ python3 error_handling_demo.py test.txt \
  http://localhost:8080
File found: test.txt
URL OK: http://localhost:8080 (status: 200)
All checks passed!
$ echo $?
0
```

6. Exit Codes Reference

Exit Code	Meaning
0	Success (all checks passed)
1	File not found
2	Permission denied
3	Unexpected file error
4	Network error or timeout
5	Unexpected error

7. Practice

- Run the script with a valid file and working URL.
- Test with a missing file.
- Test with a file that you do not have permission to read.
- Test with a refused connection (e.g. localhost on an unused port).
- Test with a timeout (e.g. a non-routable IP address).
- Check the exit code after each run using echo $?.
- Modify the script to handle additional exceptions.

8. Important Notes

- Do not hide errors with empty except blocks.
- Always log or print a clear error message.
- Use meaningful exit codes so CI/CD jobs can detect failures.
- Set timeouts to prevent scripts from hanging.
- Differentiate between expected errors (e.g. file not found) and unexpected errors (e.g. programming bugs).

9. Key Takeaways

- Use try/except to handle errors gracefully.
- Set timeouts for external requests.
- Use sys.exit(1) to signal failure to a CI/CD job.
- Never ignore or hide errors.
- Provide clear and meaningful error messages.

10. DevOps Connection

- Exit codes allow deployment pipelines to detect failures.
- Proper error handling helps create reliable automation scripts.
- Timeouts prevent jobs from hanging in CI/CD.
- Clear error messages make troubleshooting faster.
- Robust scripts reduce downtime and improve operational reliability.



Script 16: Replace Print Debugging with Useful Logging

logging_demo.py

1. What This Script Teaches

- Introduces Python's logging module for better operational messages.
- Shows log levels (DEBUG, INFO, WARNING, ERROR).
- Adds timestamps and readable log formats.
- Replaces print debugging with structured logging.
- Demonstrates logging to both console and a file.
- Teaches best practices and security considerations.



2. Key Concepts

1. **logging** – Python's built-in module for application logging.
2. **Log levels** – DEBUG, INFO, WARNING, ERROR, CRITICAL.
3. **Timestamps** – show when events happened.
4. **Handlers** – send logs to console, files, or other systems.
5. **Useful messages** – clear, relevant, non-sensitive information for troubleshooting.

3. Log Levels and When to Use

Log Level	When to Use
DEBUG	Detailed information for debugging (development).
INFO	Normal operation messages (e.g. start, success).
WARNING	Something unexpected happened, but the script can continue.
ERROR	A serious problem that prevents a task from completing.
CRITICAL	A major failure, usually application cannot continue.

4. Full Script: logging_demo.py

```python
import logging
import os
import sys
from datetime import datetime

def setup_logging(log_file="app.log", level="INFO"):
    """Configure logging to console and file."""
    log_level = getattr(logging, level.upper(), logging.INFO)

    log_format = "%(asctime)s [%(levelname)s] %(message)s"
    date_format = "%Y-%m-%d %H:%M:%S"

    # Create handlers
    console_handler = logging.StreamHandler(sys.stdout)
    console_handler.setLevel(log_level)
    console_handler.setFormatter(logging.Formatter(log_format,
                                                   datefmt=date_format))

    file_handler = logging.FileHandler(log_file)
    file_handler.setLevel(log_level)
    file_handler.setFormatter(logging.Formatter(log_format,
                                                datefmt=date_format))

    # Configure root logger
    logging.basicConfig(level=log_level, handlers=[console_handler,
                        file_handler])

    return logging.getLogger(__name__)

def main():
    logger = setup_logging()

    logger.info("Script started.")
    logger.debug("This is a debug message (only visible in DEBUG mode).")
    try:
        # Example operation
        file_path = "example.txt"
        if not os.path.exists(file_path):
            logger.warning(f"File not found: {file_path}")
        else:
            with open(file_path, "r") as f:
                content = f.read()
            logger.info(f"Successfully read {file_path}")
    except PermissionError:
        logger.error(f"Permission denied: {file_path}")
    logger.exception (f"Unexpected error: {e}")
        sys.exit(1)
    logger.info("Script completed.")
if __name__ == "__main__":
    main()
```

5. Example Console Output

```
$ python3 logging_demo.py
2026-10-21 14:32:01 [INFO] Script started.
2026-10-21 14:32:01 [WARNING] File not found: example.txt
2026-10-21 14:32:01 [INFO] Script completed.
```

Example Output (DEBUG Mode)

```
$ python3 logging_demo.py # with level=DEBUG
2026-10-21 14:33:10 [INFO] Script started.
2026-10-21 14:33:10 [DEBUG] This is a debug message (only visible in DEBUG mode).
2026-10-21 14:33:10 [WARNING] File not found: example.txt
2026-10-21 14:33:10 [INFO] Script completed.
```

Example Log File (app.log)

```
2026-10-21 14:32:01 [INFO] Script started.
2026-10-21 14:32:01 [WARNING] File not found: example.txt
2026-10-21 14:32:01 [INFO] Script completed.
```

6. Practice

- Run the script and observe the console output.
- Change the log level to DEBUG and run again.
- Create a test file (example.txt) and run the script.
- Delete or rename the file to see the WARNING message.
- Modify the script to log an ERROR by trying to read a file without permission.
- Check the generated log file (app.log).
- Add your own log messages for a simulated operation (e.g. API request, database connection).

7. Security Reminder

- Never log API tokens, passwords, secret keys, or sensitive data.
- Avoid logging full request/response bodies if they contain sensitive information.
- Be careful with environment variables that may contain secrets.
- Use masking or redaction if you must log sensitive values.

8. Key Takeaways

- Use logging instead of print for real-world scripts.
- Choose appropriate log levels (INFO, WARNING, ERROR).
- Include timestamps for better troubleshooting.
- Write clear, meaningful messages.
- Never log sensitive information.
- Logs help with monitoring, troubleshooting, and operational visibility.

9. DevOps Connection

- Structured logs make it easier to debug issues in CI/CD pipelines and production.
- Logs can be sent to centralized logging systems (e.g. cloud logging, ELK, Loki, CloudWatch).
- Proper logging improves observability, incident response, and team collaboration.



Script 17: Test Automation Before Trusting It

test_config_validator.py

1. What This Script Teaches

- Uses Python's built-in unittest to write automated tests.
- Uses unittest.mock to test success, failure, and edge cases without touching real systems.
- Tests the JSON validator with valid input, a missing field, an incorrect type, and malformed JSON.
- Shows how automated tests reduce bugs and improve confidence before deployment.
- Helps you test code safely without using production resources.



2. Key Concepts

1. **unittest** – Python's built-in testing framework.
2. **Test cases** – test expected behavior and edge cases.
3. **unittest.mock** – simulate files, responses, and external dependencies.
4. **Assertions** – verify results using assertEqual, assertRaises, etc.
5. **Isolated testing** – run tests without touching production systems.
6. **Confidence** – automated tests help you catch issues early.

3. JSON Validator (Simplified)

```
import json
REQUIRED_KEYS = ["name", "version",
  "port"]
def validate_config(data):
  if not isinstance(data, dict):
    raise ValueError("Invalid
     configuration format")
  for key in REQUIRED_KEYS:
    if key not in data:
      raise ValueError(f"Missing
       required key: {key}")
if not isinstance(data["port"], int):
  raise ValueError("Port must
   be an integer")
if not (1 <= data["port"] <= 65535):
  raise ValueError("Port must be
    between 1 and 65535")
  return True
```

4. Full Test Script: test_config_validator.py

```
import unittest
from unittest.mock import mock_open, patch
import json
from config_validator import validate_config

class TestJSONValidator(unittest.TestCase):

  def test_valid_config(self):
    data = {"name": "myapp", "version": "1.0.0", "port": 8080}
    self.assertTrue(validate_config(data))

  def test_missing_field(self):
    data = {"name": "myapp", "port": 8080} # missing version
    with self.assertRaises(ValueError) as cm:
      validate_config(data)
    self.assertIn("Missing required key: version", str(cm.exception))

  def test_incorrect_type(self):
    data = {"name": "myapp", "version": "1.0.0", "port": "eighty"}
    with self.assertRaises(ValueError) as cm:
      validate_config(data)
    self.assertIn("Port must be an integer", str(cm.exception))

  def test_malformed_json(self):
    invalid_json = "{ \"name\": \"myapp\", \"version\": 1.0.0 }"
    # malformed JSON (missing quotes around 1.0.0)
    with self.assertRaises(json.JSONDecodeError):
      json.loads(invalid_json)
if __name__ == "__main__":
  unittest.main()
```

5. Run the Tests

```
$ python3 -m unittest
....
----------------------------------------
----------------------------------------
Ran 4 tests in 0.003s

OK
```

6. Example Failure Output (Missing Field)

```
$ python3 -m unittest
.F..
========================================
FAIL: test_missing_field
----------------------------------------
Traceback (most recent call last):
  ...
ValueError: Missing required key: version
----------------------------------------
Ran 4 tests in 0.004s

FAILED (failures=1)
```

7. Example Failure Output (Incorrect Type)

```
$ python3 -m unittest
..F..
========================================
FAIL: test_incorrect_type
----------------------------------------
ValueError: Port must be an integer
----------------------------------------
Ran 4 tests in 0.003s
FAILED (failures=1)
```

8. Example Failure Output (Malformed JSON)

```
$ python3 -m unittest
...F
========================================
FAIL: test_malformed_json
----------------------------------------
json.decoder.JSONDecodeError: Expecting value: line 1 column 36 (char 35)
----------------------------------------
Ran 4 tests in 0.003s
FAILED (failures=1)
```

9. Practice

- Run the tests and confirm all tests pass.
- Modify the JSON validator and re-run the tests.
- Add a new test for an invalid port range (e.g. 70000).
- Use unittest.mock to test reading a config file without creating a real file.
- Observe how tests catch issues before deployment.

10. Security Reminder

- Never log or include API tokens, passwords, or secret keys in tests or logs.
- Use mock data instead of real credentials.
- Keep test data safe and avoid exposing sensitive information.
- Sanitize or mask sensitive values if they must appear in test output.

11. Key Takeaways

- Use unittest to validate your scripts.
- Test success, failure, and edge cases.
- Use unittest.mock to avoid touching production systems.
- Automated tests reduce bugs and increase confidence.
- Never include secrets in tests or logs.
- Test before you deploy and trust your code.

Script 18: Run Python Scripts Automatically with Cron and GitHub Actions

run_with_cron_and_github_actions.py

1. What This Script Teaches

- Shows how to run Python scripts on a schedule using cron.
- Explains working directories, Python interpreter paths, and environment differences.
- Demonstrates running the same script automatically with GitHub Actions.
- Runs the configuration validator and fails when the configuration is invalid.
- Highlights the importance of logs and failure notifications for scheduled jobs.



2. Key Concepts

1. **cron** – time-based job scheduler on Linux.
2. **Working directory** – use absolute paths to avoid path issues.
3. **Python interpreter** – specify the full path (e.g. **/usr/bin/python3**).
4. **Environment differences** – cron has a limited environment (no user profile, different PATH).
5. **GitHub Actions** – run scripts in CI/CD pipelines on push, pull request, or schedule.
6. **Logs and notifications** – essential to troubleshoot failures.

3. Cron Example

Edit your crontab:

```
$ crontab -e
# Run the config validator every day
# at 6:00 AM. Use absolute paths.
0 6 * * * /usr/bin/python3 \
/home/user/scripts/config_validator.py \
--config /home/user/config/app.json \
>> /home/user/logs/config_check.log 2>&1
```

Important Notes:

- Use absolute paths (both Python and script).
- Set the correct working directory.
- Redirect output to a log file.
- Cron has a limited environment, so set any required environment variables inside the script or in the cron command.

4. GitHub Actions Workflow

Create **.github/workflows/config-check.yml**:

```yaml
name: Configuration Check

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 6 * * *' # run daily at 6 AM UTC

jobs:
  validate-config:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies (if any)
        run: |
          python -m pip install --upgrade pip

      - name: Run configuration validator
        run: |
          python scripts/config_validator.py \
            --config config/app.json
```

5. Example: Run Manually in Terminal

```
$ python3 scripts/config_validator.py \
  --config config/app.json
Configuration is valid!
Exit code: 0
```

6. Example: Failing Run (Invalid Config)

```
$ python3 scripts/config_validator.py \
  --config config/invalid.json
Error: Missing required key: version
Exit code: 1
```

7. Example: Cron Log Output

```
$ tail -n 5 /home/user/logs/config_check.log
2026-10-21 06:00:01 [INFO] Starting
configuration validation...
2026-10-21 06:00:02 [ERROR] Missing
required key: version
2026-10-21 06:00:02 [ERROR] Validation
failed with exit code 1
```

8. Exit Codes

Exit Code	Meaning	Use in Automation
0	Success	Job continues
1	Validation failed	Fail the pipeline or cron job
2	Unexpected error	Alert and investigate

9. Practice

- Set up a cron job to run the configuration validator daily.
- Create a GitHub Actions workflow to run the same validator.
- Test with a valid configuration file.
- Test with an invalid configuration file and confirm the job fails.
- Check the logs and understand the failure message.
- (Optional) Add a notification step (e.g. email or Slack) when the job fails.

10. Important Notes

- Scheduled jobs need logs so you can troubleshoot issues.
- Failure notifications help you respond quickly.
- Cron runs with a minimal environment – always use absolute paths.
- CI/CD pipelines prevent bad configuration from reaching production.
- A failing job should exit with a non-zero code (e.g. sys.exit(1)).

11. Key Takeaways

- Automate script execution with cron and GitHub Actions.
- Use absolute paths and proper environments.
- Validate configuration files before deployment.
- Logs and notifications are essential for reliability.
- Automated validation catches issues early.

Script 19: Final Project: Build a Mini DevOps Operations Toolkit

mini_devops_toolkit/

1. What This Project Does

- Brings the 10 scripts into one Git repository.
- Organizes scripts, sample data, tests, and documentation.
- Shows how to run each check independently.
- Keeps failures isolated so one script does not stop all others.
- Avoids destructive automation (no deleting or changing real systems).
- Generates a summary report of all checks.
- Provides a beginner-friendly, reproducible project.

2. Project Structure

```
mini_devops_toolkit/
├── scripts/
│   ├── 01_large_file_finder.py
│   ├── 02_log_analyzer.py
│   ├── 03_system_info.py
│   ├── 04_health_check.py
│   ├── 05_backup_checker.py
│   ├── 06_service_monitor.py
│   ├── 07_config_validator.py
│   ├── 08_docker_status.py
│   ├── 09_inventory_report.py
│   └── 10_deployment_verifier.py
├── data/
│   ├── sample_config.json
│   └── inventory.csv
├── tests/
│   └── test_config_validator.py
└── docs/
    ├── README.md
    └── USAGE.md
└── requirements.txt (only if needed)
```

3. Run Individual Scripts

Run any script independently:

```
# Find large files
python3 scripts/01_large_file_finder.py \
  --path /var/log --min-mb 100

# Validate configuration
python3 scripts/07_config_validator.py \
  --config data/sample_config.json

# Check Docker containers
python3 scripts/08_docker_status.py
```

Each script:

- Has its own input and options.
- Can run independently.
- Handles its own errors.
- Does not stop other scripts from running.

4. Run All Checks and Generate a Summary Report

Run a simple runner script (run_all.py) to execute all 10 scripts and create a summary report:

```
python3 run_all.py --report reports/summary.txt
```

Example output:

```
Mini DevOps Toolkit - Summary Report
====================================

01_large_file_finder.py      ✅ PASSED
02_log_analyzer.py           ✅ PASSED
03_system_info.py            ✅ PASSED
04_health_check.py           ✅ PASSED
05_backup_checker.py         ✅ PASSED
06_service_monitor.py        ❌ FAILED (1 issue)
07_config_validator.py       ✅ PASSED
08_docker_status.py          ✅ PASSED
09_inventory_report.py       ✅ PASSED
10_deployment_verifier.py    ✅ PASSED

------------------------------------

Total: 9 passed, 1 failed
Report saved to: reports/summary.txt
```

5. Testing

Run automated tests (example):

```
python3 -m unittest discover tests/ -v
```

Tests include:

- Valid configuration input.
- Missing required field.
- Incorrect type.
- Malformed JSON.

6. Requirements File

Only create requirements.txt if you use external libraries. Example:

```
# requirements.txt
requests>=2.31.0
pytest>=7.0.0  # if using pytest instead
               # of unittest (optional)
```

Most scripts use Python's built-in modules, so no external dependencies are required.

7. Best Practices

- Keep each script focused on one task.
- Use clear input arguments.
- Handle errors gracefully.
- Keep failures isolated (one script failing should not stop others).
- Avoid destructive actions (no deleting or modifying real data).
- Write clear logs and a summary report.
- Use meaningful exit codes.
- Document how to run everything.
- Make the project easy to reproduce on any machine.

8. Deliverable

A beginner-friendly, reproducible GitHub repository containing:

- All 10 scripts.
- Sample data files.
- Automated tests.
- Documentation.
- A simple runner script and summary report.
- Clear instructions in the README.

9. README.md (Key Sections)

- Project overview.
- List of all 10 scripts and what they do.
- Requirements (if any).
- How to run each script.
- How to run all checks.
- How to run tests.
- Sample output.
- Safety notes (no destructive actions).
- Troubleshooting tips.

10. Key Takeaways

- Organize multiple scripts in a single repository.
- Keep code, data, tests, and documentation structured.
- Run checks independently or all together.
- Handle failures without stopping other scripts.
- Automated tests increase confidence.
- A clear summary report helps you understand the health of your systems.
- A well-documented project is easy to share and reproduce.

11. DevOps Connection

- Real DevOps teams build toolkits like this to perform routine operational checks.
- Helps with monitoring, compliance, audits, and change planning.
- Supports automation, reliability, and faster troubleshooting.
- Demonstrates how small scripts can create big operational value when combined.

Script 20: Your First DevOps Automation Challenge

devops_automation_challenge/

1. Scenario Overview

A newly deployed application is returning errors, and disk usage is rising on the server.

Your Goals:

1. Use the appropriate scripts to gather evidence.
2. Identify symptoms vs. root causes.
3. Determine the safest next step.
4. Explain your reasoning.
5. Avoid destructive actions.



2. Symptoms and Evidence

A. Application Errors (Logs)

```
$ tail -n 10 /var/log/app.log
2026-10-21 10:15:23 [ERROR] Failed to
connect to database: timeout
2026-10-21 10:15:24 [ERROR] 500
Internal Server Error
2026-10-21 10:15:25 [ERROR] Retrying
in 5 seconds...
```

B. Disk Usage (Rising)

```
$ df -h
Filesystem  Size  Used  Avail  Use%  Mounted on
/dev/sda1   50G   46G   2.1G   96%   /
tmpfs       2.0G  200M  1.8G   10%   /dev/shm
```

C. Running Containers

```
$ docker ps --format '{{.Names}} {{.Status}}'
web       Up 10 minutes
api       Up 10 minutes
db        Up 10 minutes
```

3. Recommended Scripts

Choose and run the relevant scripts from your toolkit:



1. **04_health_check.py** — Check application health.
2. **01_large_file_finder.py** — Find large files using disk space.
3. **08_docker_status.py** — Inspect container states.
4. **09_inventory_report.py** — Check environment and host details.
5. **10_deployment_verifier.py** — Verify the deployment and app version.

4. Example Commands and Output

A. Check Application Health

```
$ python3 scripts/04_health_check.py \
  --url http://localhost:8080/health
Status: FAIL ❌
Response time: 12.5s
Error: Connection timeout
```

B. Find Large Files

```
$ python3 scripts/01_large_file_finder.py \
  --path /var/log --min-mb 100
Found 2 large files:
/var/log/app.log    2.4 GB
/var/log/old.log    1.1 GB
```

C. Check Docker Containers

```
$ python3 scripts/08_docker_status.py
NAME      STATE       STATUS
web       running     Up 10 minutes
api       running     Up 10 minutes
db        running     Up 10 minutes
```

D. Verify Deployment

```
$ python3 scripts/10_deployment_verifier.py \
  --url http://localhost:8080 \
  --expected-version 1.0.0
Version check: PASS ✅
Health check: FAIL ❌
Deployment status: UNHEALTHY
```

5. Investigate and Analyze

1. Is the issue application-related or disk-related?
2. Are the errors causing large logs?
3. Are containers healthy?
4. Is the database reachable?
5. Do you see old or rotated log files that can be archived (not deleted without approval)?
6. What changed recently (deployment, configuration, environment)?

6. Determine the Safest Next Step

- ✅ Collect more information if needed.
- ✅ Archive or rotate old log files (not delete without approval).
- ✅ Check database connectivity.
- ✅ Verify application configuration.
- ✅ Consider restarting a specific service only if necessary and approved.
- ✅ Document your findings and recommend the next action.

7. Important Reminders

- ⚠ Do not delete files or make destructive changes without approval.
- ⚠ Avoid exposing sensitive information in logs or output.
- ⚠ Distinguish between symptoms (e.g. high disk usage) and root causes (e.g. endless error logs).
- ⚠ Always consider the impact on production users before taking action.

8. Assessment Questions

1. Which scripts would you run first? Why?
2. What does the health check script tell you about the application?
3. How do you confirm whether disk usage is caused by large log files?
4. How would you handle a script failure (e.g. permission error)?
5. What exit code would indicate failure? How should a CI/CD pipeline respond?
6. What security considerations apply when running these scripts?
7. When should you not automate a fix?
8. Based on the evidence, what is the safest next step and why?

9. Final Outcome

- ✅ You can explain the issue clearly.
- ✅ You can execute the relevant scripts.
- ✅ You can troubleshoot and interpret the results.
- ✅ You can recommend and test a safe next step.
- ✅ You can improve the scripts if needed (e.g. better error handling or logging).

10. Key Takeaways

- ✅ Use multiple sources of evidence.
- ✅ Automate checks, not destructive actions.
- ✅ Logs and disk usage can reveal hidden problems.
- ✅ Verify before making changes.
- ✅ Clear documentation helps teams respond faster.
- ✅ You are now ready to build and extend real DevOps automation solutions.